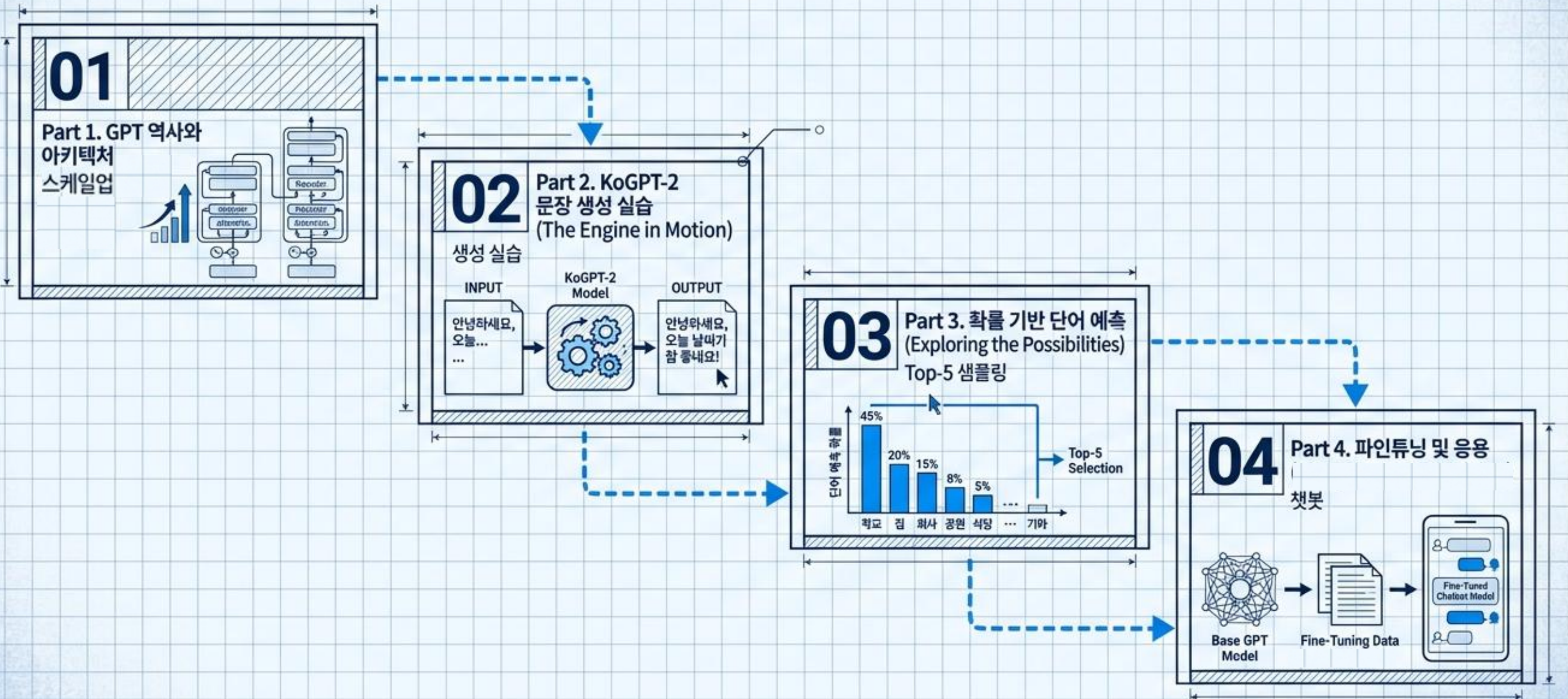


GPT의 구조와 문장 생성

Generative Pre-trained Transformer

- 발표 주제: GPT(Generative Pre-trained Transformer)의 내부 구조 및 문장 생성 원리
- 주요 내용: GPT 아키텍처, KoGPT-2 실습, Top-5 샘플링, 파인튜닝 응용

설계도 목차



01

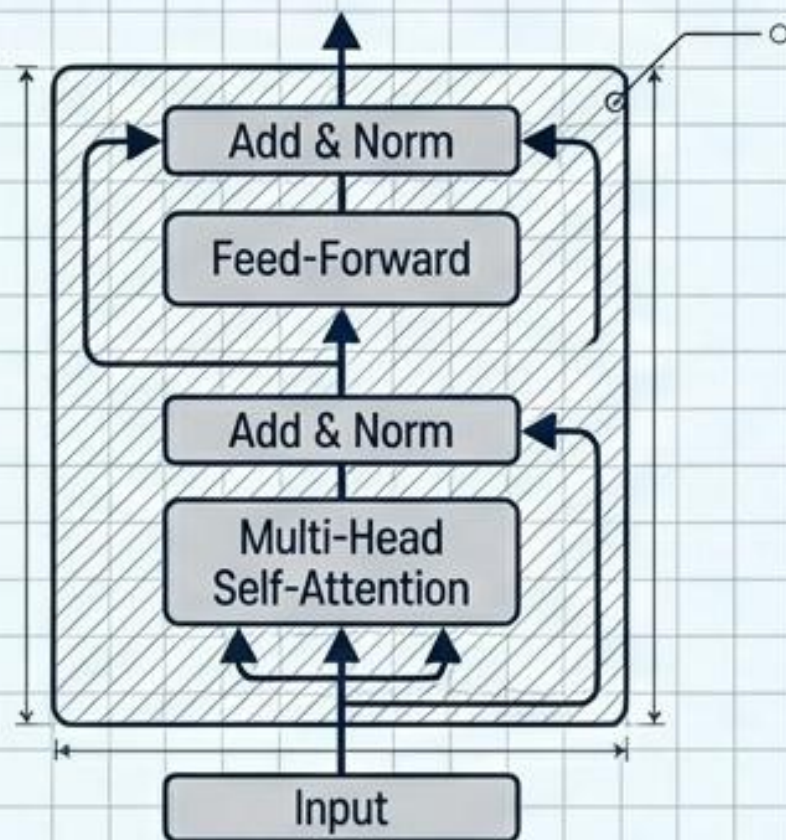
GPT 역사와 아키텍처

- 트랜스포머 아키텍처에서의 디코더(Decoder) 분리
- 파라미터 스케일업을 통한 성능 진화의 역사
- 생성 모델(Generative Model)로서의 근본적 특성 파악

아키텍처의 분기: BERT vs GPT

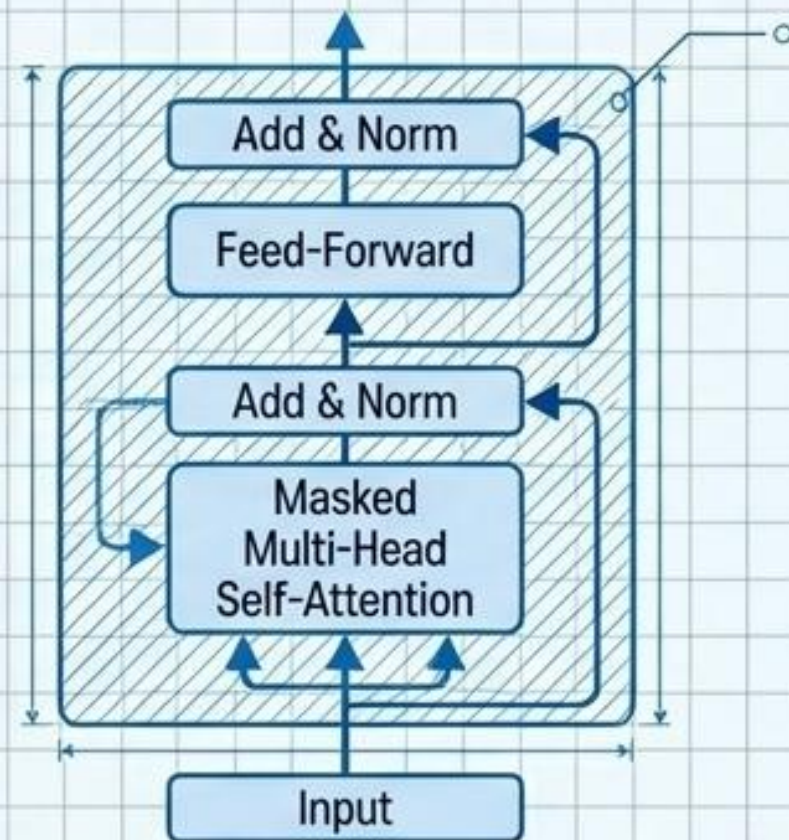
공통점: 트랜스포머(Transformer) 아키텍처에서 파생된 핵심 딥러닝 모델

BERT (인코더 Only)



- 트랜스포머의 '인코더(Encoder)' 부분만 분리하여 설계
- 주요 목적: 양방향 문맥 파악을 통한 '언어 이해(Language Understanding)'

GPT (디코더 Only)



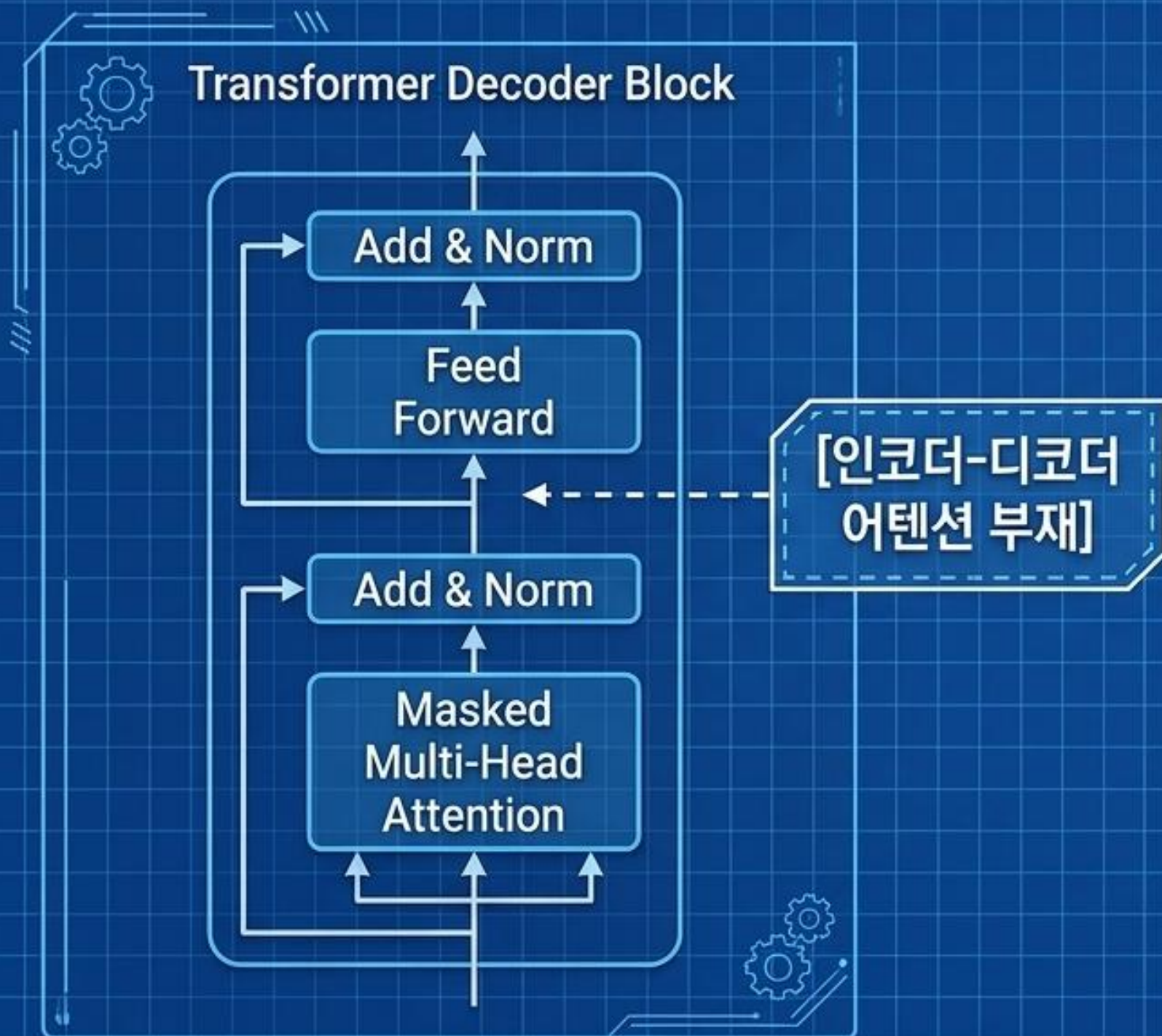
- 트랜스포머의 '디코더(Decoder)' 부분만 분리하여 설계
- 주요 목적: 이전 단어로 다음 단어를 예측하는 '언어 생성(Language Generation)'

파라미터 폭발과 스케일업의 역사

- OpenAI는 2018년 GPT-1부터 모델 파라미터 크기를 기하급수적으로 확장
- GPT-3는 GPT-1 대비 1,400배, GPT-2 대비 117배 파라미터 증가
- 거대한 스케일업이 언어 모델의 문맥 이해 및 지시 수행 능력(Zero-shot) 향상으로 직결

모델	파라미터 개수	디코더 층	토큰 개수	엔닉층 크기
GPT-1	1억 1,700만 개	12	512	768
GPT-2	15억 개	48	1,024	1,600
GPT-3	1,750억 개	96	2,048	12,288
GPT-3.5	비공개	비공개	비공개	비공개
GPT-4	1조 8천억 개 (추정)	비공개	128,000	비공개

GPT 아키텍처: 디코더 Only 구조



- 오직 '트랜스포머 디코더' 층으로만 모델이 구성됨
- 초기 트랜스포머 구조에 존재했던 '인코더-디코더 어텐션' 층이 완전히 제거됨
- 이전 단어들을 기반으로 다음 단어를 예측하는 구조이므로 지속적인 글쓰기(생성) 가능
- GPT-1부터 최신 거대 언어 모델(LLM)까지 근본적인 아키텍처의 불변성 유지

02

The Engine in Motion: KoGPT-2 문장 생성 실습

- Hugging Face transformers 라이브러리 기반 환경 셋팅
- 한국어 모델(KoGPT-2) 맵핑 및 프롬프트 정의
- 정수 인코딩부터 텍스트 디코딩까지의 전체 생성 파이프라인 구현

모델 엔진 로드 (Model & Tokenizer Mapping)

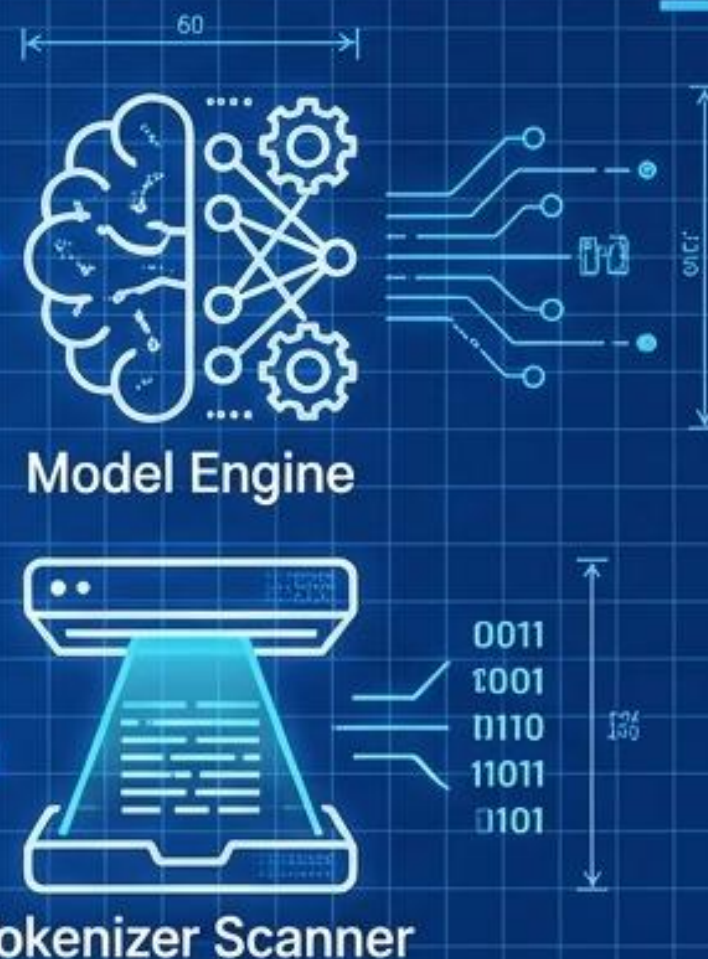
패키지 임포트

```
from transformers import AutoTokenizer, GPT2LMHeadModel
```

모델 및 토크나이저 인스턴스화

```
model = GPT2LMHeadModel.from_pretrained('skt/kogpt2-base-v2')
```

```
tokenizer = AutoTokenizer.from_pretrained('skt/kogpt2-base-v2')
```



- 사전 학습된 모델을 불러오기 위해 **transformers 라이브러리** 사용
- 모델과 토크나이저는 반드시 **동일한 이름의 맵핑(Mapping)** 관계를 가져야 함
- 한국어 텍스트 생성을 위해 SKT가 공개한 **skt/kogpt2-base-v2** 사용

프롬프트 주입 및 정수 인코딩 (Tokenization)

```
# 입력 프롬프트 정의  
sent = '근육이 커지기 위해서는'  
  
# 정수 인코딩 수행  
input_ids = tokenizer.encode(sent, return_tensors='pt')  
print(input_ids)
```

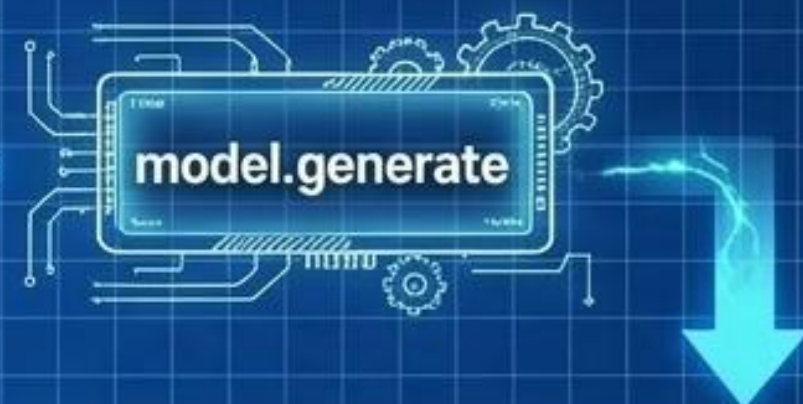
‘근육이 커지기 위해서는’



tensor([[33245, 10114, 12748, 11357]])

- 생성할 문장의 **방향성(Context)**을 제공하기 위해 **초기 프롬프트 문자열 정의**
- 텍스트를 GPT가 연산할 수 있는 **텐서(Tensor)** 형태의 정수 시퀀스로 변환하는 과정
- 공백과 형태소 기준으로 텍스트가 여러 개의 **고유 정수 토큰**으로 분해됨

■ □ △



- 138

디코딩 및 생성 결과 확인 (Decoding)

```
tokenizer.decode(output_ids)
```

“ ‘근육이 커지기 위해서는 [무엇보다 규칙적인 생활습관이 중요하다. 특히, 아침식사는 단백질과 비타민이 풍부한 과일과 채소를 많이 섭취하는 것이 좋다. 또한 하루 30분 이상 충분한 수면을 취하는 것도 도움이 된다... (중략)]’ ”

- 생성된 정수 시퀀스를 토큰라이저 내부의 **Vocabulary** 맵핑 테이블을 역참조하여 한국어로 복원
- 주어진 문맥에 맞춰 건강 및 생활습관과 관련된 글을 매우 자연스럽게 생성해 냄
- 주의: 언어 모델의 특성상 그럴듯해 보이지만 **사실 여부(Hallucination)** 검증은 반드시 필요함



Exploring the Possibilities: 확률 기반 단어 예측 (Top-5 샘플링)

- 다음 단어 예측(Next Word Prediction)의 원리
- 모델 출력값(Logits)의 구조 분석
- Top-K 샘플링을 통한 문장 생성의 다양성 및 랜덤성 부여

로짓(Logits) 추출: 단어 집합에 대한 확률 분포

```
output = model(input_ids)
logits = output.logits[0, -1]
```

```
print(logits.shape)
> torch.Size([51200])
```

문맥의 마지막 단어
(Input)



- 언어 모델은 문맥 다음에 올 단어로 '확률이 가장 높은' 단어(Top 1)를 기본적으로 선택함
- 모델 내부적으로는 단어 집합 전체(약 51,200개)에 대해 확률 분포 벡터(Logits)를 계산하여 순위를 매김
- `model()`을 직접 호출하여 최종 레이어의 출력값(Logits)을 추출 및 확인 가능

Top-5 후보 단어 추출 및 디코딩

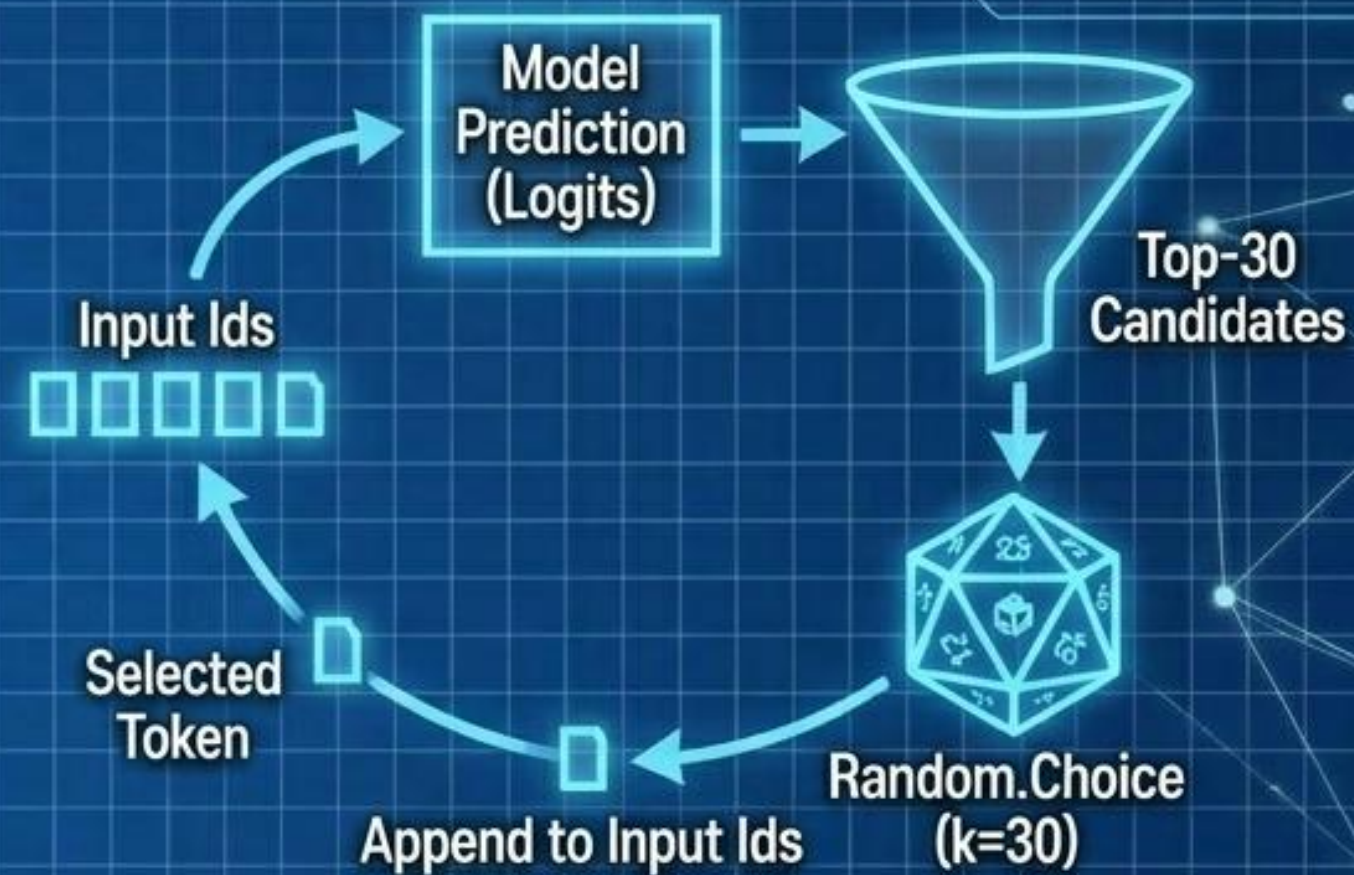
```
top5 = torch.topk(logits, k=5)
tokens =
tokens = [tokenizer.decode(
    token_id) for token_id in
    top5.indices.tolist()]
print(tokens)
```



- 파이토치의 **torch.topk** 함수를 사용하여 51,200개의 로짓 중 가장 높은 점수의 5개 인덱스 추출
- 실제 1위 예측 단어였던 '무엇보다' 외에도, 문맥에 어울리는 유력한 대기 단어들을 확인
- 때로는 '피부'와 같이 다소 엉뚱하지만 문법적으로 허용 가능한 단어도 상위권에 랭크됨

다양성 확보: Top-K 다이나믹 샘플링 루프

```
while len(input_ids[0]) < 50:  
    output = model(input_ids)  
    logits = output.logits[0, -1]  
    top5 = torch.topk(logits, k=30) # 상위 30개 후보  
    token_id = random.choice(top5.indices.tolist())  
    input_ids = torch.cat(...) # 기존 입력에 이어 붙임
```



- 단순 Top 1 추출(Greedy Search)의 한계: 매번 동일하고 기계적인 텍스트만 생성됨
- 해결책: 매 시점(Time Step)마다 상위 K개 단어 중 '랜덤'으로 하나를 선택하여 다음 단어로 이어 붙임
- 이를 통해 매번 새로운 문맥을 창조하며 예측 불가능한 다양성(Temperature/Top-K 효과) 달성

샘플링 기반 생성 결과: 새로운 문맥의 탄생

Top-1 디코딩 (이전 결과)

‘근육이 커지기 위해서는 무엇보다 규칙적인 생활습관이 중요하다...’

Top-K 샘플링 결과

‘근육이 커지기 위해서는 반드시 필요한 것이 건강이다. 이처럼 피부과 의사들은 피부 노화에 가장 많이 노출되어 있는 피부 질환으로 노화로 인하여 생기는 주름의 종류를 ‘눈밑, 팔자, 미간’ 순으로 분류한다. 특히, ‘주름’을 가진 눈은

- 상위 후보들 중 ‘반드시’가 선택되며, 이전 예제(운동/생활습관)와 완전히 다른 ‘피부와 및 노화’ 관련 이야기로 전개됨
- 생성형 인공지능이 창의적이고 다양한 답변을 도출해내는 핵심 메커니즘

GPT-2를 이용한 한국어 뉴스 감성 분류

Pre-trained Language Model 기반의 금융 뉴스 긍정/부정 분류 파이프라인 구축

Text Generation 모델을 Sequence Classification 엔진으로 전환하는 완벽한 실무 가이드

프로젝트 목표 및 핵심 파이프라인

프로젝트 목표

- 한국어 금융 뉴스 문장의 감성(긍정/중립/부정)을 자동으로 분류하는 인공지능 모델 개발
- OpenAI의 GPT-2 아키텍처 기반 모델(skt/kogpt2-base-v2) 활용
- 생성형(Generative) 모델을 분류형(Classification) 작업으로 전이 학습(Fine-tuning)

파이프라인 아키텍처

- Phase 1: 비정형 텍스트 데이터 수집 및 탐색 (EDA)
- Phase 2: 텐서 변환 및 어텐션 마스크(Attention Mask)를 포함한 전처리 엔진 구축
- Phase 3: 손실 함수 최소화 및 모델 가중치 업데이트를 위한 학습 루프 설계
- Phase 4: 수학적 지표(Accuracy, F1) 기반의 성능 검증
- Phase 5: HuggingFace Pipeline API를 활용한 실시간 추론(Inference) 모듈 배포



실습 환경 셋팅 및 핵심 라이브러리 로드

환경 구성 (Environment Setup)

- Google Colab 환경 권장 및 하드웨어 가속기 'GPU' 할당 필수 (Tesla T4 등)
- HuggingFace 생태계 활용을 위한 transformers, datasets 패키지 설치

핵심 라이브러리 импорт

- 데이터 처리: Pandas, Numpy 등 기본 패키지 호출
- 딥러닝 엔진: PyTorch 및 HuggingFace Transformers 모듈 호출
- 평가 지표: Scikit-learn의 모델 검증 및 메트릭 패키지 호출

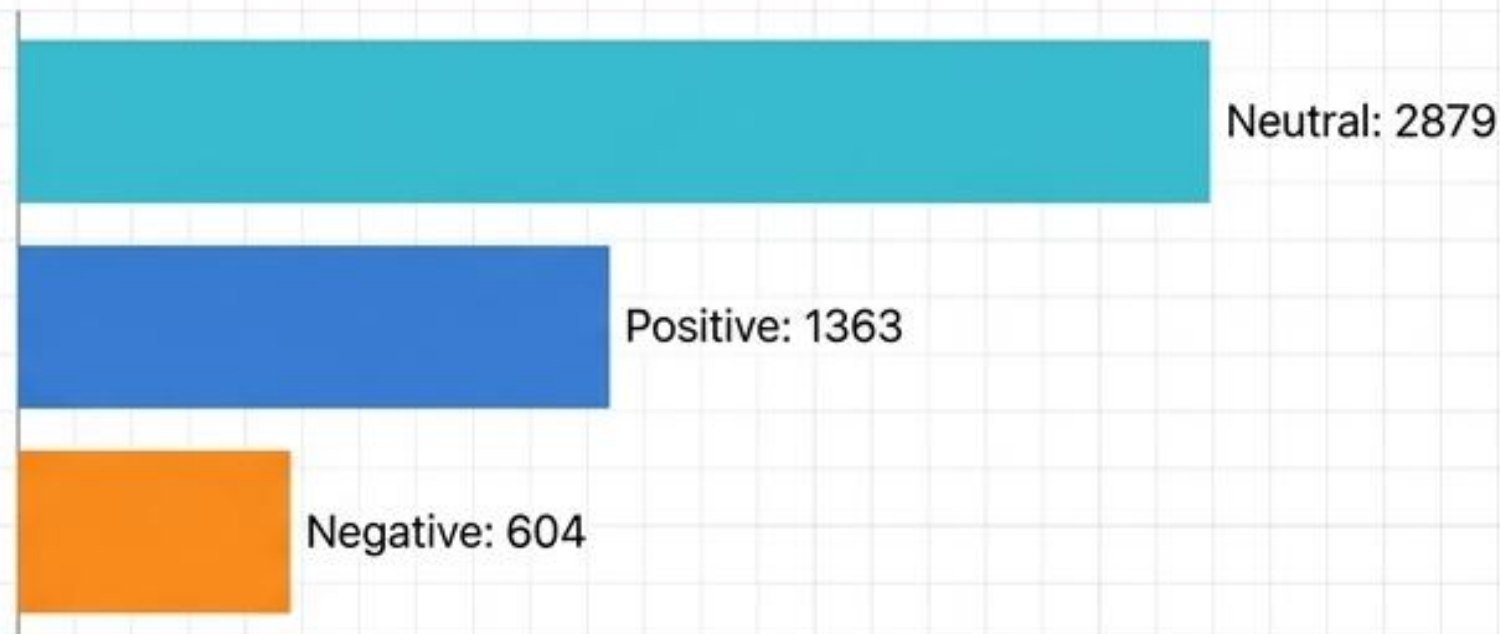
```
!pip install transformers datasets

from datasets import load_dataset
import pandas as pd
import numpy as np
import torch
from transformers import AutoTokenizer,
AutoModelForSequenceClassification
from torch.utils.data import TensorDataset,
DataLoader, RandomSampler,
SequentialSampler
from sklearn.model_selection import
train_test_split
from sklearn.metrics import f1_score,
accuracy_score
```


데이터 수집 및 감성 레이블 분포 분석

금융 뉴스 데이터셋 다운로드

- GitHub에 공개된 `finance_sentiment_corpus` 활용
- 총 4,846개의 금융 관련 뉴스 문장으로 구성된 CSV 파일



클래스 분포 분석 (Class Imbalance)

- 중립(neutral): 2,879개 (가장 큰 비중)
- 긍정(positive): 1,363개
- 부정(negative): 604개 (데이터 불균형 확인)

```
!wget
https://raw.githubusercontent.com/ukairia777/
finance_sentiment_corpus/main/finance_data.csv

df = pd.read_csv('finance_data.csv')
print('샘플의 개수 :', len(df))
df['labels'].value_counts()
```

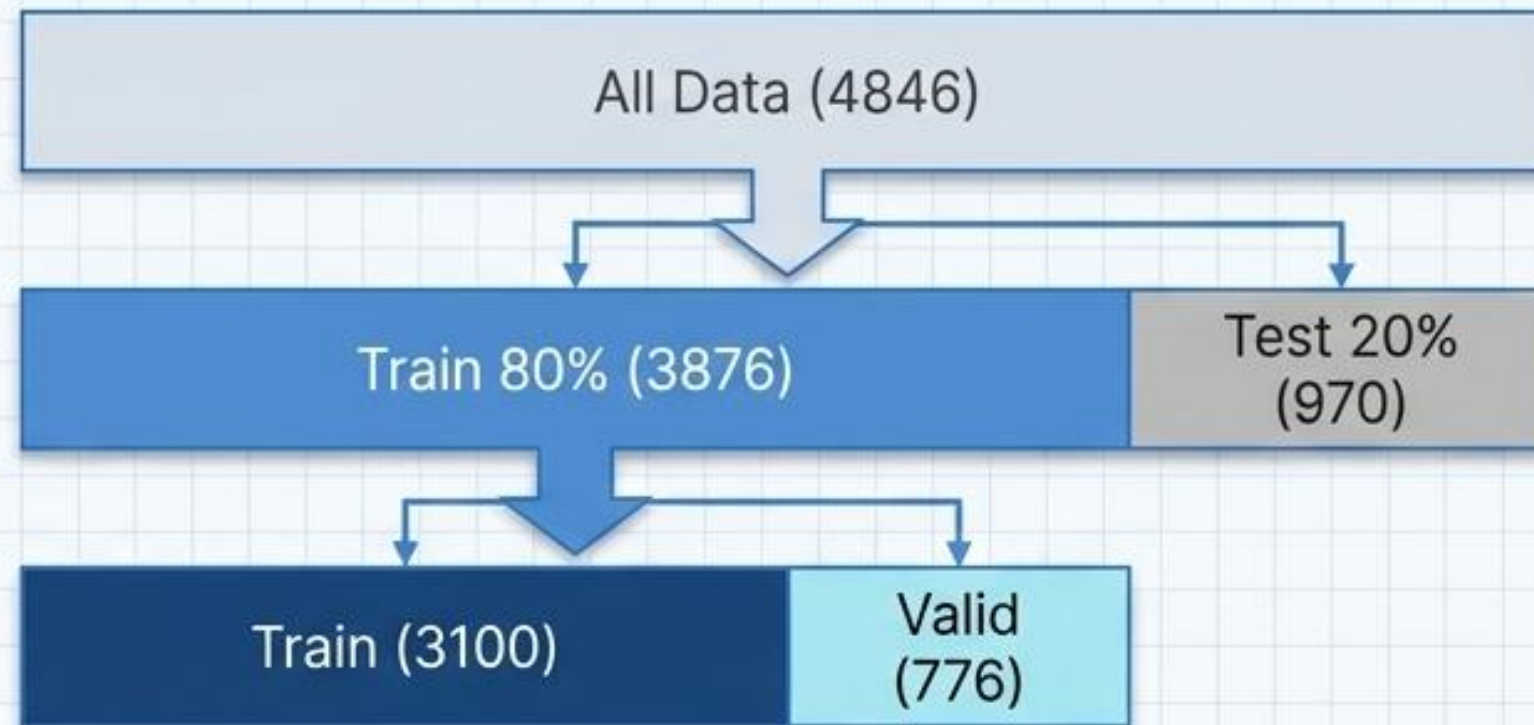

레이블 인코딩 및 Dataset 분할

정수 인코딩 (Label Encoding)

- 머신러닝 연산을 위해 문자열 레이블을 정수형 텐서 값으로 맵핑
- 변환 규칙: neutral -> 0, positive -> 1, negative -> 2

HuggingFace Dataset 변환 및 분할

- datasets 패키지의 load_dataset을 이용하여 최적화된 포맷으로 로드
- 1차 분할: Train (80%) / Test (20%)
- 2차 분할: Train 데이터를 다시 Train / Validation(20%)으로 분할
- 최종 데이터 수: Train 3,100개 / Valid 776개 / Test 970개

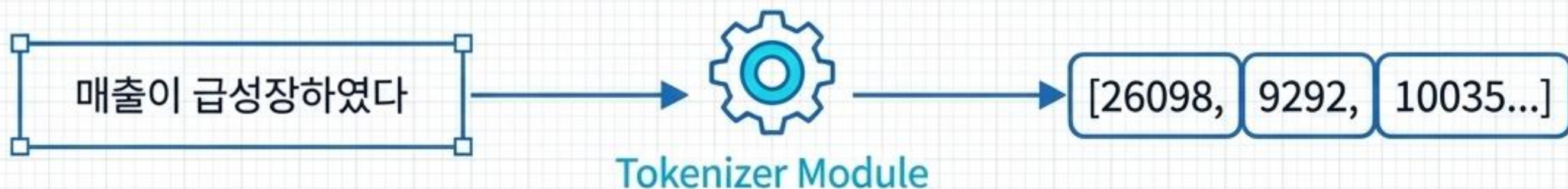


```
df['labels'] = df['labels'].replace(['neutral', 'positive', 'negative'], [0, 1, 2])
all_data = load_dataset("csv", data_files={"train": "finance_data.csv"})

# 1차 분할
cs = all_data['train'].train_test_split(0.2, seed=777)
train_cs, test_cs = cs["train"], cs["test"]

# 2차 분할 (Valid 분리)
cs = train_cs.train_test_split(0.2, seed=777)
train_cs, valid_cs = cs["train"], cs["test"]
```


GPT-2 토크나이저 (Tokenizer) 로드



모델 종속성 (Model Dependency)

- 사전 학습 모델과 토크나이저는 반드시 동일한 맵핑 관계를 유지해야 함
- 내부 Vocabulary를 통해 텍스트를 모델이 이해할 수 있는 정수(Integer)로 변환

SKT KoGPT-2 토크나이저

- 한국어 텍스트에 최적화된 skt/kogpt2-base-v2 토크나이저 활용
- 본 전처리 단계에서 텍스트 시퀀스 열(kor_sentence)을 리스트로 추출 후 토큰화 진행

```
# 리스트 변환 (Train, Valid, Test 동일 적용)
train_sentences = list(train_cs['kor_sentence'])
train_labels = train_cs['labels']

# 사전 학습된 한국어 GPT-2 토크나이저 로드
tokenizer = AutoTokenizer.from_pretrained('skt/kogpt2-base-v2')
```


텍스트 전처리 핵심 개념: 패딩 (Padding)

패딩 (Padding)의 필요성

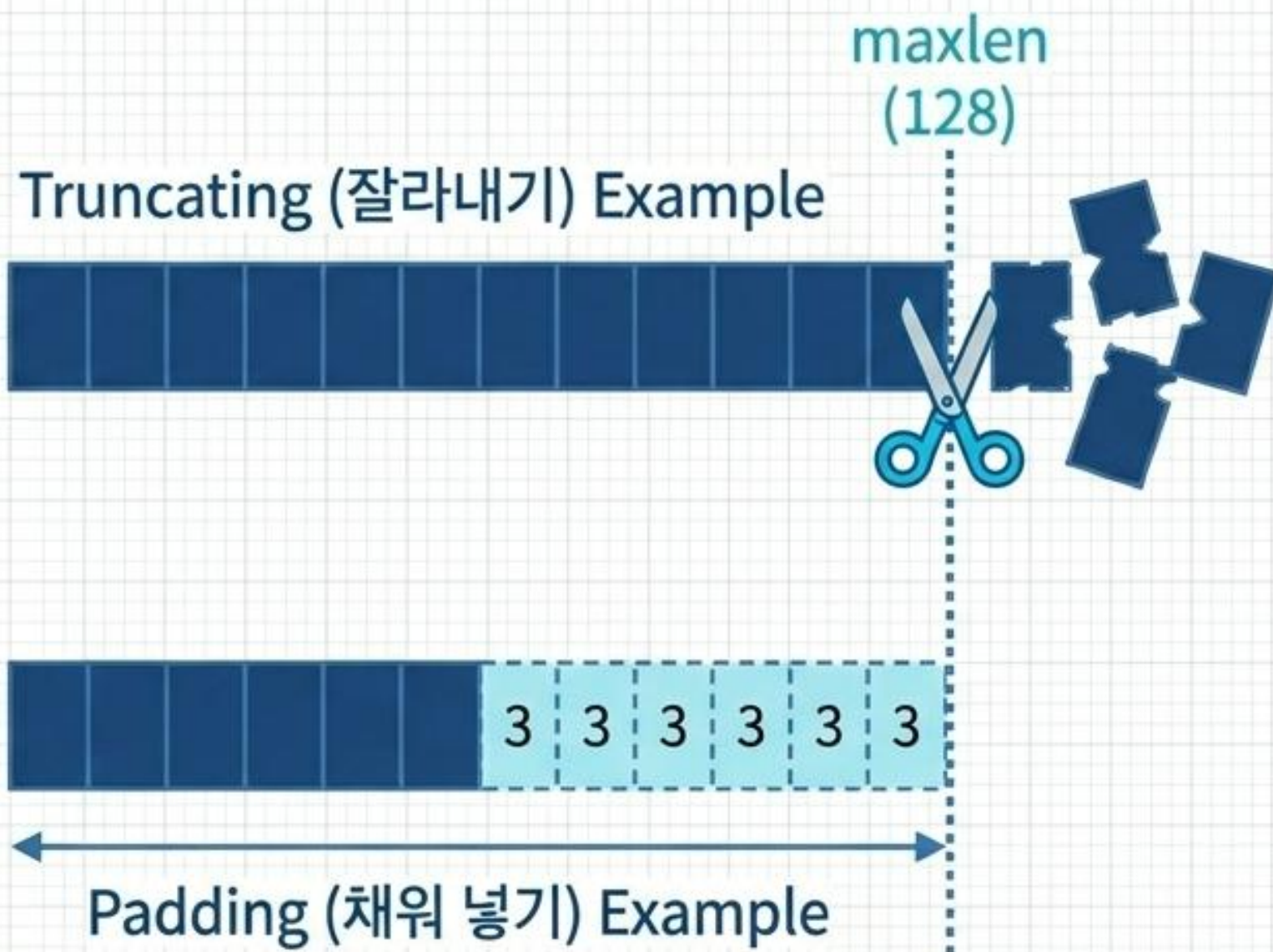
- 텍스트 데이터는 길이가 각기 다르지만, 모델의 행렬(Tensor) 연산은 동일한 길이를 요구함
- 가장 긴 문장 혹은 지정된 길이(maxlen)에 맞춰 짧은 문장 뒤에 무의미한 토큰(Pad token)을 채워 넣음

트렁케이팅 (Truncating)

- 메모리 한계를 넘는 지나치게 긴 문장은 지정된 최대 길이에 맞춰 초과분을 잘라냄

NLP 표준 적용 방향

- 자연어 처리는 문장의 핵심 내용이 훼손되지 않도록 뒤쪽에 패딩을 넣는 padding='post' 방식 선호
- KoGPT-2 모델의 경우 패딩 토큰 ID는 3으로 할당됨



[코드 해부] pad_sequences 함수 구현

- 초기화 로직

- 최대 길이(maxlen)를 파악하고 입력된 value값으로 가득 찬 빈 Numpy 배열(N-dimensional Array) 생성

- 데이터 삽입 로직

- 조건에 따라 긴 데이터는 자르고(truncating), 짧은 데이터는 채움(padding)
- 정수 시퀀스들을 안전하게 저장하기 위해 dtype='long' 타입 캐스팅 사용

```
def pad_sequences(sequences, maxlen=None, dtype='int32',
                  padding='pre', truncating='pre', value=0.0):
    if not sequences: return np.array([])
    if maxlen is None: maxlen = max(len(seq) for seq in sequences)

    if dtype == 'long':
        result = np.full((len(sequences), maxlen), value,
                        dtype=np.int64)
    else:
        result = np.full((len(sequences), maxlen), value,
                        dtype=dtype)

    for i, seq in enumerate(sequences):
        if len(seq) == 0: continue
        # Truncating
        if len(seq) > maxlen:
            if truncating == 'pre': seq = seq[-maxlen:]
            else:
                seq = seq[:maxlen]
        # Padding
        if padding == 'pre': result[i, -len(seq):] = seq
        else: result[i, :len(seq)] = seq
    return result
```

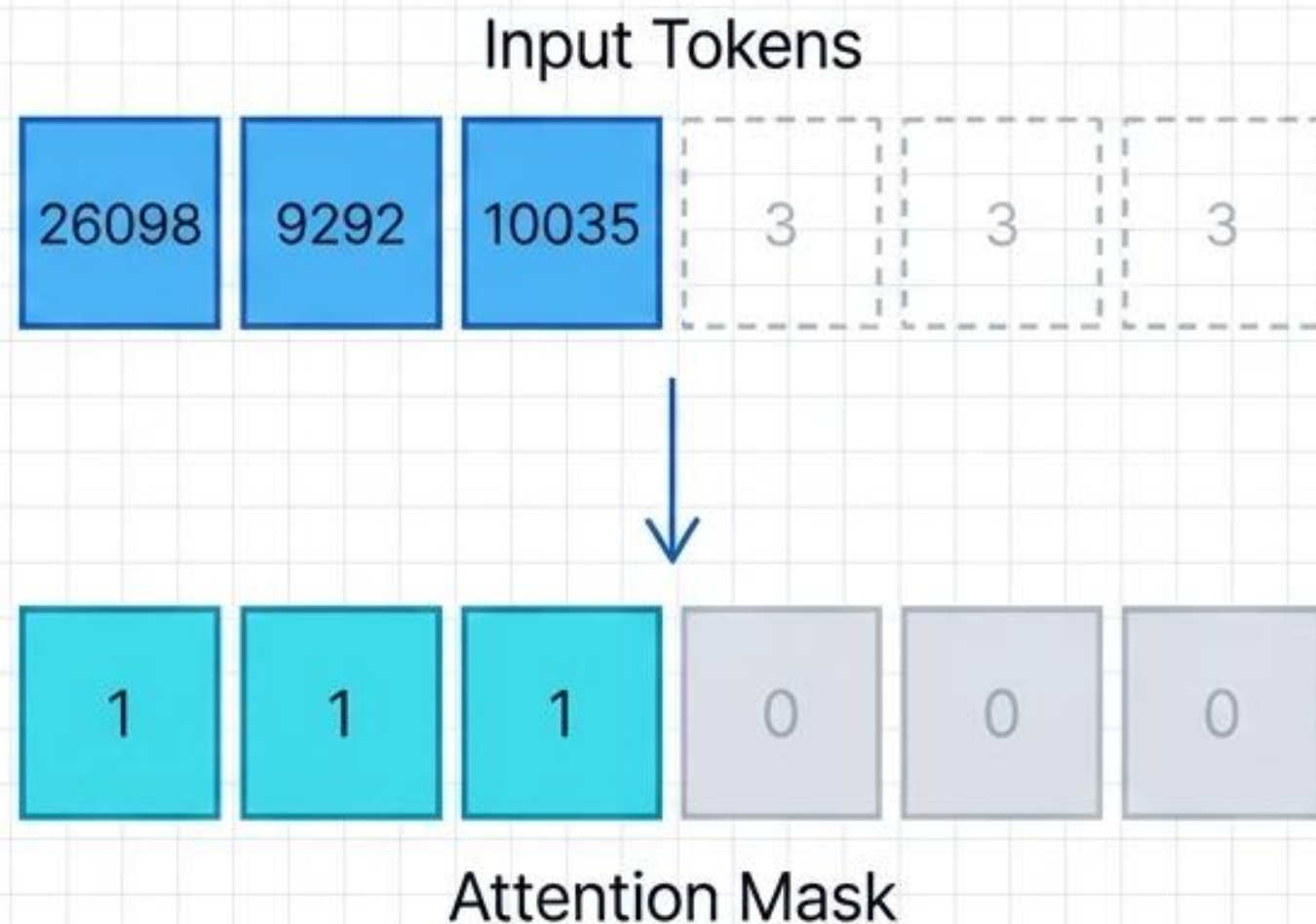

텍스트 전처리 핵심 개념: Attention Mask

어텐션 마스크 (Attention Mask)의 목적

- 모델이 실제 유의미한 텍스트 데이터와 무의미한 패딩(Padding) 토큰을 구분하도록 알려주는 지도(Map) 정보
- Self-Attention 연산 시 패딩 토큰이 가중치 계산에 영향을 미치지 않도록 차단

작동 방식 및 맵핑

- 실제 텍스트 데이터가 존재하는 위치: 1 (연산 포함)
- 패딩 토큰이 존재하는 위치: 0 (연산 제외)
- 불필요한 연산을 방지하여 메모리 효율을 높이고 모델 학습의 정확도 극대화



[코드 해부] data_to_tensor 함수 구현

■ 통합 전처리 파이프라인

- 토큰화(Tokenization) + 정수 인코딩 + 패딩 + 어텐션 마스크 생성을 한 번에 수행하는 통합 함수

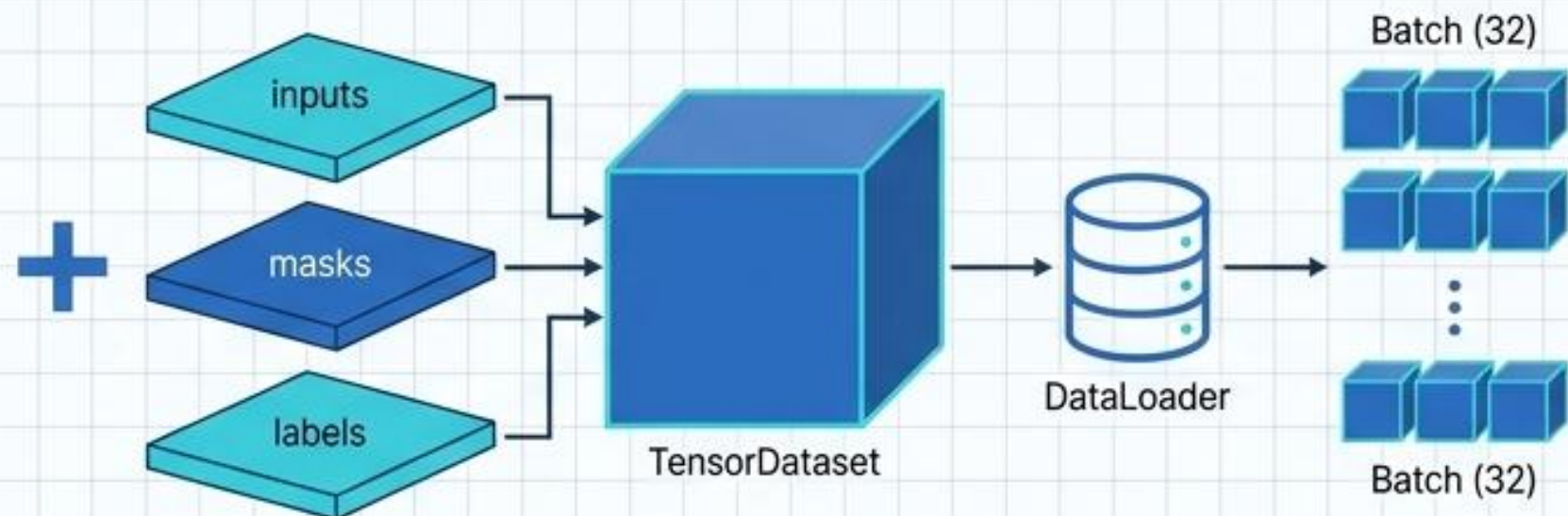
■ 최대 길이 제한 및 텐서화

- GPU 메모리 효율을 위해 최대 시퀀스 길이(MAX_LEN)를 128로 제한
- 모든 리스트 데이터를 PyTorch 학습에 호환되는 torch.tensor 구조로 최종 변환

```
max_len = 128
```

```
def data_to_tensor(sentences, labels, MAX_LEN):  
    tokenized_texts = [tokenizer.tokenize(sent) for  
                        sent in sentences]  
    input_ids = [tokenizer.convert_tokens_to_ids(x) for  
                 x in tokenized_texts]  
  
    pad_token = tokenizer.encode('<pad>')[0]  
    input_ids = pad_sequences(input_ids, maxlen=MAX_LEN,  
                             value=pad_token, dtype='long',  
                             truncating='post', padding='post')  
  
    attention_masks = []  
    for seq in input_ids:  
        seq_mask = [float(i != pad_token) for i in seq]  
        attention_masks.append(seq_mask)  
  
    tensor_inputs = torch.tensor(input_ids)  
    tensor_labels = torch.tensor(labels)  
    tensor_masks = torch.tensor(attention_masks)  
    return tensor_inputs, tensor_labels, tensor_masks
```


텐서 적용 및 데이터 배치화 (Batching) 설계



전처리 함수 일괄 적용

- 정의한 `data_to_tensor` 함수를 Train, Valid, Test 데이터에 일괄 적용하여 inputs, masks, labels 추출

TensorDataset 및 미니 배치 (Mini-batch) 생성

- 분리되어 있던 입력, 마스크, 레이블을 하나의 세트(TensorDataset)로 결합
- 한 번에 모든 데이터를 학습할 수 없으므로 배치 사이즈 32로 분할

Sampler 설계 전략

- Train 데이터: 과적합(Overfitting) 방지를 위해 RandomSampler로 무작위 섞음(Shuffle) 적용
- Valid/Test 데이터: 평가 일관성을 위해 SequentialSampler로 데이터 순서 유지

```
# Train Data 적용 예시
train_inputs, train_labels, train_masks = data_to_tensor(train_sentences, train_labels, max_len)

batch_size = 32
train_data = TensorDataset(train_inputs, train_masks, train_labels)
train_sampler = RandomSampler(train_data)
train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=batch_size)
```

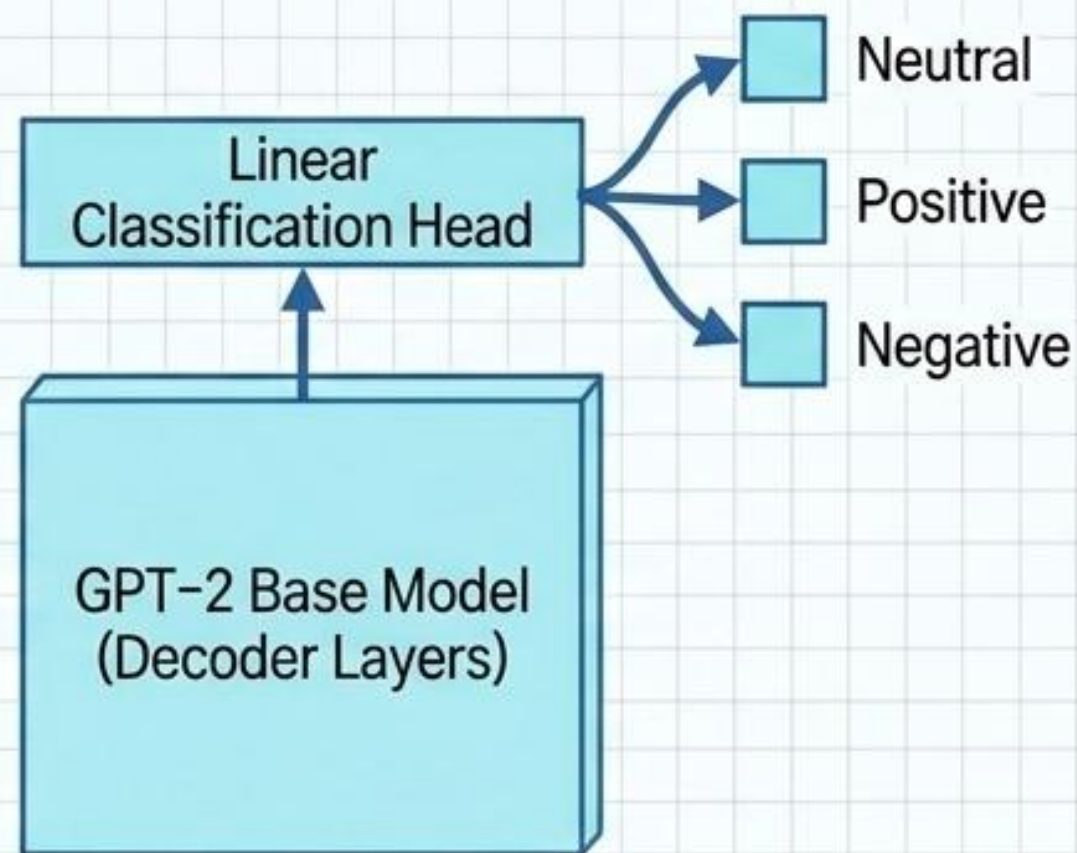

디바이스 설정 및 모델 로드 (Sequence Classification)

GPU 가속기 할당 (CUDA)

- 대규모 트랜스포머 파라미터 연산을 위해 GPU 디바이스 설정 필수
- Colab 환경의 하드웨어 가속기(예: Tesla T4) 자동 감지 로직 적용

목적 맞춤형 아키텍처 로드

- 텍스트 생성(Text Generation) 목적의 모델을 문장 분류(Classification) 작업으로 전환
- AutoModelForSequenceClassification 클래스 활용
- 기존 GPT-2 디코더 상단에 3개 클래스를 구분하는 분류기(Linear Layer) 부착 (num_labels=3)



```
if torch.cuda.is_available():
    device = torch.device("cuda")
    print('We will use the GPU:', torch.cuda.get_device_name(0))
else:
    device = torch.device("cpu")

num_labels = 3
model = AutoModelForSequenceClassification.from_pretrained("skt/kogpt2-base-v2", num_labels=num_labels)
model.cuda()
```


Fine-tuning 하이퍼파라미터 (Hyperparameters) 셋팅

학습 에포크 (Epochs)

3

- 전체 데이터셋을 반복 학습할 횟수
- 과도한 에포크는 Catastrophic Forgetting이나 과적합을 유발하므로 제한

옵티마이저 (Optimizer)

AdamW

- 가중치 업데이트를 위한 최적화 알고리즘
- L2 정규화를 분리하여 적용하는 AdamW(Weight Decay 포함) 채택

학습률 (Learning Rate)

$2e-5$

- 미세 조정에 적합한 매우 작은 단위의 학습률 (0.00002)
- 언어 모델의 지식을 파괴하지 않고 분류 헤드의 가중치만 세밀하게 조정

```
epochs = 3
from torch.optim import AdamW

# 모델의 파라미터와 학습률을 옵티마이저에 전달
optimizer = AdamW(model.parameters(), lr= $2e-5$ )
```


모델 평가 지표 (Mathematical Metrics)

불균형 데이터 평가 전략

- 단순 정확도(Accuracy)만으로는 데이터 불균형(중립 압도적 다수)으로 인한 착시가 발생할 수 있음
- Precision(정밀도)과 Recall(재현율)을 모두 고려하는 F1-Score 동시 측정

$$\begin{aligned} &\text{정확도 (Accuracy)} \\ &\text{전체 예측 중 모델이 정답을 맞춘 비율} \\ &Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \\ &\text{F1-Score} \\ &\text{정밀도와 재현율의 조화 평균(Harmonic Mean)} \\ &F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \end{aligned}$$

다중 클래스 평균 산출 방식 (Averaging)

- **Macro**: 모든 클래스에 동일한 가중치 부여 (소수 클래스 성능 확인에 유리)
- **Micro**: 전체 데이터의 개수를 기준으로 산출
- **Weighted**: 각 클래스의 데이터 수(Support)를 가중치로 적용

커스텀 평가 함수 로직 구현

다중 평가 지표 일괄 계산

- Scikit-learn(sklearn.metrics) 패키지를 활용하여 앞서 정의한 수학적 지표들을 함수화
- 예측값(Predictions)과 실제 정답(Labels) 리스트를 입력받아 딕셔너리 형태로 결과 반환

Zero Division 처리

- 특정 클래스의 예측이 전혀 발생하지 않을 경우 분모가 0이 되는 오류를 방지하기 위해 zero_division=0 파라미터 명시

```
def metrics(predictions, labels):  
    y_pred = predictions  
    y_true = labels  
    accuracy = accuracy_score(y_true, y_pred)  
  
    # 다중 클래스 F1 스코어 계산  
    f1_macro = f1_score(y_true=y_true, y_pred=y_pred,  
                        average='macro', zero_division=0)  
    f1_micro = f1_score(y_true=y_true, y_pred=y_pred,  
                       average='micro', zero_division=0)  
    f1_weighted = f1_score(y_true=y_true, y_pred=y_pred,  
                           average='weighted', zero_division=0)  
  
    return {'accuracy': accuracy,  
            'f1_macro': f1_macro,  
            'f1_micro': f1_micro,  
            'f1_weighted': f1_weighted}
```

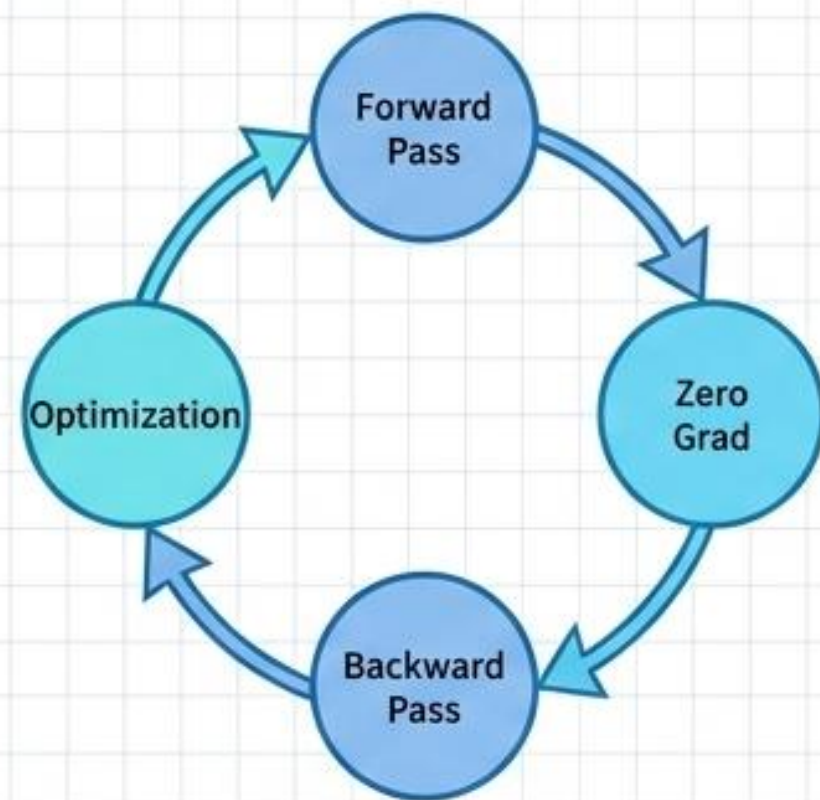

[코드 해부] 학습 루프 설계 (train_epoch)

상태 전환 및 1 에포크 궤적

- model.train() 모드로 전환하여 드롭아웃(Dropout) 등 학습 전용 레이어 활성화
- 데이터로더에서 미니 배치를 순차적으로 꺼내며 반복 진행

학습의 4단계 생명주기

1. Forward Pass: 입력값을 모델에 통과시켜 예측값 산출 및 손실(Loss) 계산
2. Zero Grad: 이전 스텝의 기울기 누적을 방지하기 위한 초기화
3. Backward Pass: 역전파(loss.backward())를 통한 기울기(Gradient) 계산
4. Optimization: 계산된 기울기를 바탕으로 가중치 업데이트 (optimizer.step())



```
def train_epoch(model, train_dataloader, optimizer, device):  
    total_train_loss = 0  
    model.train()  
    for step, batch in tqdm(enumerate(train_dataloader), desc="Training Batch"):  
        batch = tuple(t.to(device) for t in batch)  
        b_input_ids, b_input_mask, b_labels = batch  
  
        outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask, labels=b_labels)  
        loss = outputs.loss  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
        total_train_loss += loss.item()  
    return total_train_loss / len(train_dataloader)
```


[코드 해부] 검증 루프 설계 (evaluate)

평가 모드 전환

- model.eval() 모드로 전환하여 파라미터 업데이트를 멈추고 순수 예측 성능만 측정

연산 최적화 (no_grad)

- 기울기(Gradient) 계산 기능을 비활성화(torch.no_grad())하여 메모리 사용량을 대폭 줄이고 추론 속도 극대화

분류 예측값 결정 (Argmax)

- 모델 출력값(Logits) 3개 중 가장 수치가 큰 값의 인덱스를 찾아 최종 클래스로 확정

```
def evaluate(model, validation_dataloader, device):  
    model.eval()  
    total_eval_loss = 0  
    predictions, true_labels = [], []  
    for batch in validation_dataloader:  
        batch = tuple(t.to(device) for t in batch)  
        b_input_ids, b_input_mask, b_labels = batch  
        with torch.no_grad():  
            outputs = model(b_input_ids, token_type_ids=None,  
                             attention_mask=b_input_mask, labels=b_labels)  
            total_eval_loss += outputs.loss.item()  
            logits = outputs.logits.detach().cpu().numpy()  
            label_ids = b_labels.to('cpu').numpy()  
  
            # 최고 확률 클래스 선택  
            predictions.extend(np.argmax(logits, axis=1).flatten())  
            true_labels.extend(label_ids.flatten())  
  
    eval_metrics = metrics(predictions, true_labels)  
    return total_eval_loss / len(validation_dataloader), eval_metrics
```


[코드 해부] 메인 학습 실행 및 체크포인트 (Checkpoint) 저장

■ 메인 실행 컨트롤러

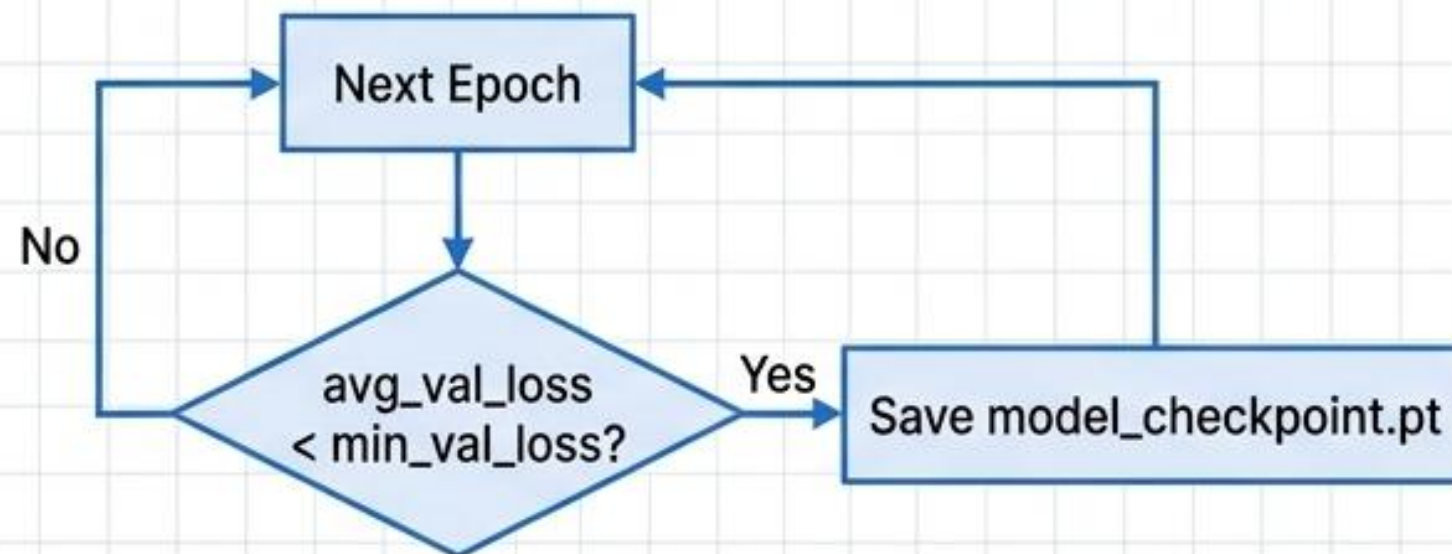
- 사전에 정의된 에포크(epochs=3)만큼 train_epoch과 evaluate 함수를 순차적으로 반복 실행

■ 조기 종료 및 베스트 모델 모니터링

- 매 에포크마다 검증 오차(Val Loss)를 확인하여 모델의 과적합(Overfitting) 여부 판별
- 검증 오차가 이전 최소값(min_val_loss)을 갱신할 경우 최적의 상태로 간주

■ 물리적 저장 (State Dict)

- 가장 오차가 적은 최적점의 모델 가중치 (Weights)를 model_checkpoint.pt 파일로 물리적 저장



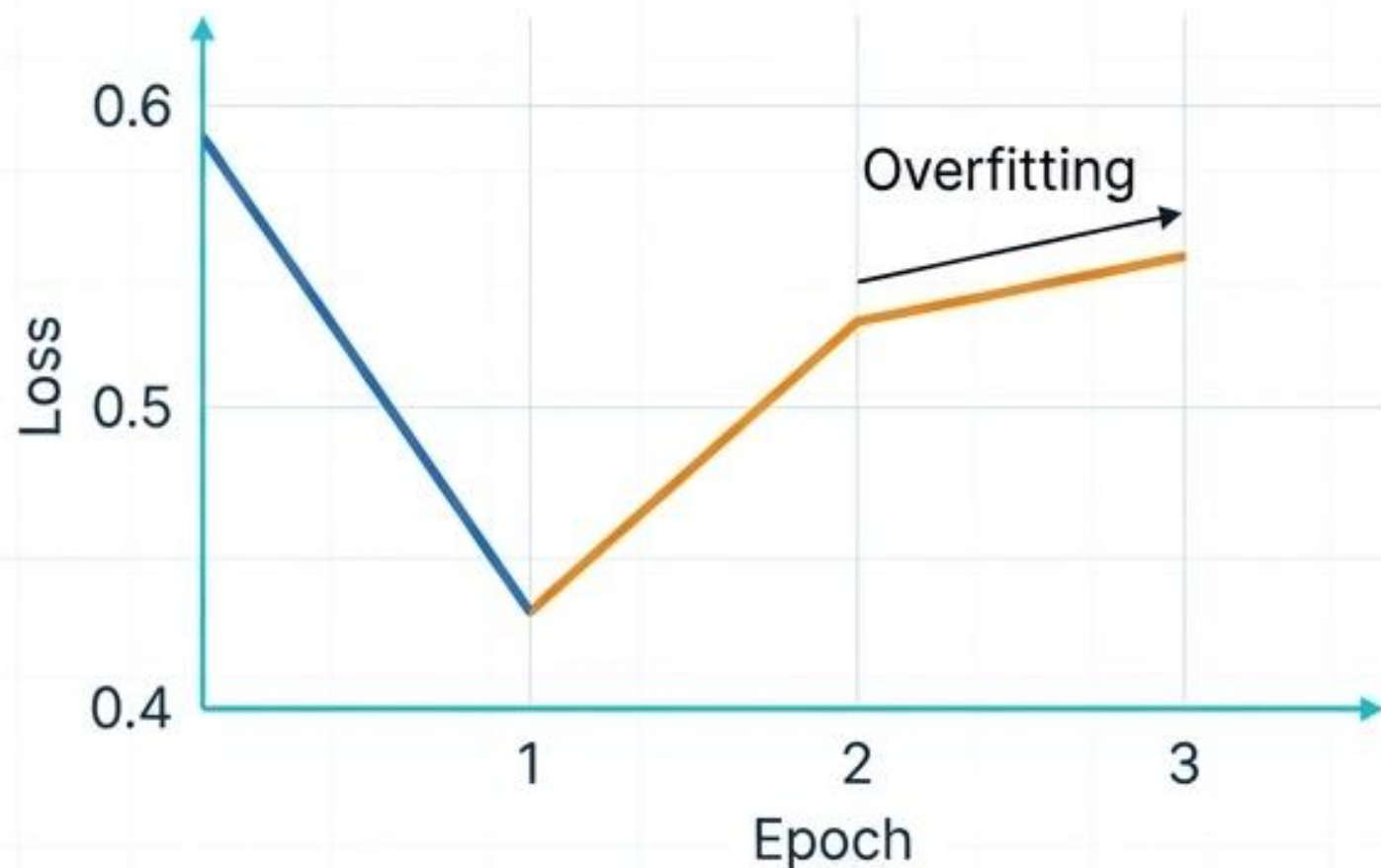
```
min_val_loss = float('inf')
for epoch_i in range(0, epochs):
    # 1. 학습
    train_epoch(model, train_dataloader, optimizer, device)
    # 2. 검증
    avg_val_loss, eval_metrics = evaluate(model, validation_dataloader, device)

    # 3. 모델 저장 (Validation 손실 감소 시)
    if avg_val_loss < min_val_loss:
        print(f"Validation loss decreased. Saving model...")
        torch.save(model.state_dict(), 'model_checkpoint.pt')
        min_val_loss = avg_val_loss
```


학습 결과 모니터링 및 최종 테스트 평가

에포크(Epoch)별 학습 추이 분석

- **Epoch 1:** Loss 0.43 / Accuracy 0.84
(가장 안정적인 성능 기록, 모델 저장됨)
- **Epoch 2:** Loss 0.53 / Accuracy 0.79
(과적합 발생, Validation Loss 증가)
- **Epoch 3:** Loss 0.55 / Accuracy 0.83
(손실치 증가 추세 유지)
- **결론:** 1 에포크 만에 Fine-tuning이 최적점에 도달함을 확인



일반화 성능 검증 (Test Data)

- 저장해둔 베스트 모델(model_checkpoint.pt)을 로드하여 한 번도 본 적 없는 테스트 데이터로 최종 성능 검증
Test Loss: 0.45
Test Accuracy: 0.82 (82%)
F1 Macro: 0.79 / F1 Weighted: 0.82

실전 추론 파이프라인 (Inference Pipeline) 셋팅

HuggingFace Pipeline API

- 복잡한 텍스트 전처리(Tokenization)와 텐서 변환, 모델 주입 과정을 내부적으로 자동화하는 고수준 API
- text-classification 작업(Task)으로 명시하여 모델의 활용 목적 정의

소프트맥스 (Softmax) 확률 변환

- 원시 Logit 값을 0~1 사이의 확률값으로 변환하도록 function_to_apply='softmax' 속성 설정
- 단 2줄의 코드로 딥러닝 백엔드 없이 텍스트 입력만으로 결과를 산출 가능한 파이프라인 구축 완성

```
from transformers import pipeline

# 추론 파이프라인 생성 (가장 높은 확률의 정답 1개만 출력)
pipe = pipeline("text-classification",
                 model=model.cuda(),
                 tokenizer=tokenizer,
                 device=0,
                 max_length=512,
                 function_to_apply='softmax')

result = pipe('SK하이닉스가 매출이 급성장하였다')
print(result)
# [{'label': 'LABEL_1', 'score': 0.981047}]
```


커스텀 예측 함수 구현 및 서비스 적용 테스트

예측 래퍼(Wrapper) 함수 설계

- 기계적인 출력(LABEL_1)을 인간이 읽기 편한 한글 텍스트('긍정', '부정', '중립')로 변환하는 해시 맵 구현

Project 결론 및 확장성

- 텍스트 생성(Generative) 구조인 디코더 모델(GPT-2)을 분류(Classification) 엔진으로 성공적으로 전이 학습(Fine-tuning)
- 구축된 파이프라인은 스팸 필터링, 리뷰 감성 분석 등 타 비즈니스 도메인에 즉각 적용 가능

```
label_dict = {'LABEL_0': '중립', 'LABEL_1': '긍정', 'LABEL_2': '부정'}
```

```
def prediction(text):  
    result = pipe(text)  
    return [label_dict[result[0]['label']]]
```

```
>> prediction('네이버가 매출이 급성장하였다')  
['긍정']
```

```
>> prediction('ChatGPT의 등장으로 인공지능 스타트업들은 위기에 빠졌다. 이에 OpenAI를 제외한 기타 스타트업들의 주가가 하락하였다.')  
['부정']
```

```
>> prediction('인공지능 기술의 발전으로 누군가는 기회를 얻을 것이고, 누군가는 얻지 못할 것이다')  
['중립']
```


GPT-2 기반 챗봇 모델 파인튜닝 아키텍처

대화형 인공지능 구축을 위한
PyTorch 설계 청사진



챗봇 구축 파이프라인 청사진

- 사전 학습된 언어 모델을 활용한 대화형 에이전트 구축 프로세스
- 총 5단계의 모듈화된 조립(Modular Assembly) 공정 수행



모듈 1: KoGPT-2 기반 환경 초기화

- transformers 패키지를 통한 사전 학습 모델 로드
- 토큰나이저와 모델의 매핑 관계(Mapping Relation) 필수 유지
- 시작 토큰(bos_token), 종료 토큰(eos_token), 패딩 토큰(pad_token) 지정

```
tokenizer = AutoTokenizer.  
from_pretrained('skt/kogpt2-base-v2',  
                bos_token='</s>',  
                eos_token='</s>',  
                pad_token='<pad>')
```

```
model = GPT2LMHeadModel.  
from_pretrained('skt/kogpt2-base-v2')
```

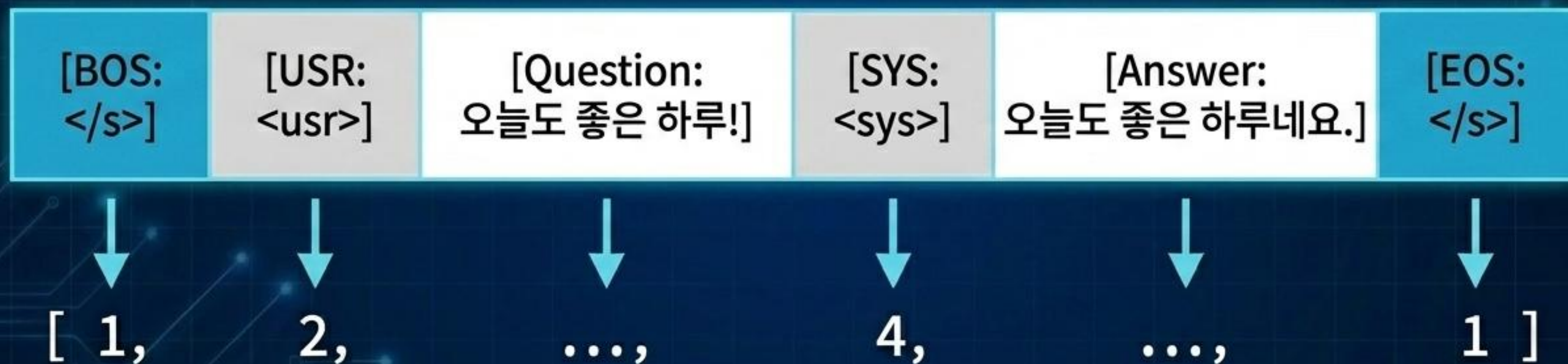

모듈 2: 특수 토큰(Special Token) 진단 매트릭스

- 일반적인 텍스트가 아닌, 모델에게 구조적 지시를 내리기 위한 예약 토큰
- 문장의 시작/끝 판별, 배치 처리를 위한 패딩, 대화 주체 구분 수행

구분(Category)	토큰 문자열(String)	정수 ID(ID)	기능(Function)
시작 및 종료 (BOS/EOS)	</s>	1	문장 경계 인식
사용자 (User)	<usr>	2	질문 주체 식별
패딩 (Padding)	<pad>	3	배치 텐서 길이 정렬
시스템 (System)	<sys>	4	챗봇 응답 주체 식별

챗봇 프롬프트 아키텍처 (데이터 구조화)

- 특수 토큰을 활용한 역할 분리(Role Separation) 샌드위치 구조
- 사용자 질문과 챗봇 응답을 하나의 단일 시퀀스로 결합하여 학습



모듈 3: 챗봇 훈련 데이터셋 확보

- 송영숙(songys) Github의 오픈소스 챗봇 대화 데이터 활용
- 질문(Q)과 답변(A) 쌍으로 이루어진 구조 확인
- 전체 학습용 대화 샘플 개수 확보 (11,823개)

Q (질문) A (답변)	label
[Q: 오늘 날씨 어때요?] [A: 맑고 화창한 날씨입니다.]	0
[Q: 근처 맛집 추천해줘.] [A: 강남역 3번 출구 앞 '맛집 식당'을 추천합니다.]	1
[Q: 최신 영화 정보 알려줘.] [A: 현재 상영 중인 영화는 '000', 'XXX'입니다.]	0

총 샘플 수:
11,823개

ChatDataset 클래스 설계도

- torch.utils.data.Dataset 상속을 통한 커스텀 데이터셋 구축
- 데이터로더(DataLoader) 연동을 위한 필수 내부 메서드 3종 정의



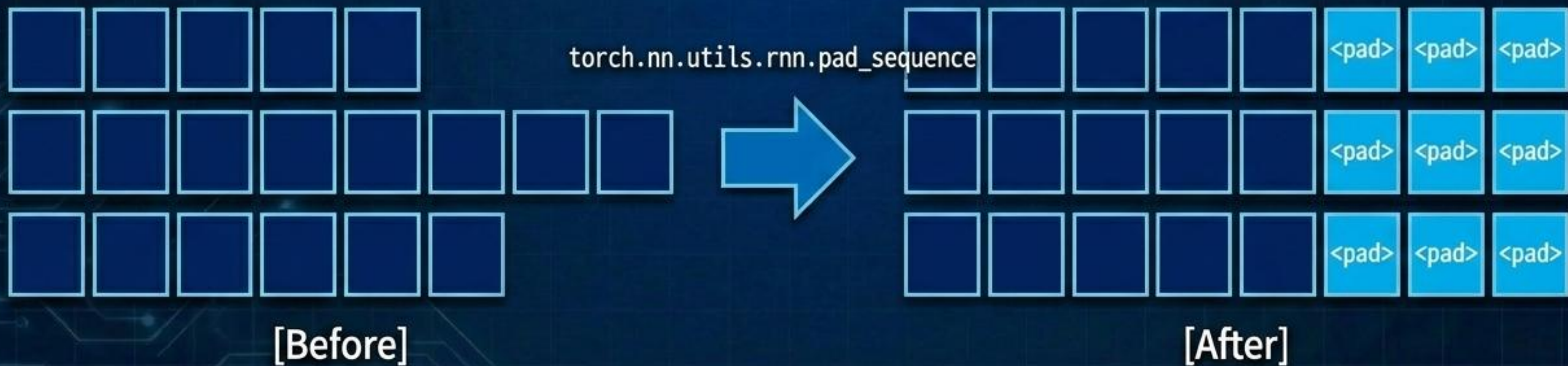
데이터 추출 및 결합 로직: `__getitem__`

- 질문(Q)과 답변(A) 텍스트를 인덱스(idx) 기반으로 추출
- 태그(<usr>, <sys>)와 결합된 문자열을 토큰라이저로 정수 인코딩
- 시작 토큰(bos_token)과 종료 토큰(eos_token)을 양 끝에 결합하여 텐서 반환

```
1: question = self.train_data.Q.iloc[idx]
2: answer = self.train_data.A.iloc[idx]
3: sent = self.tokenizer.encode('<usr>' + question + '<sys>' + answer, add_special_tokens=False)
4: return torch.tensor([bos_token] + sent + [eos_token], dtype=torch.long)
```

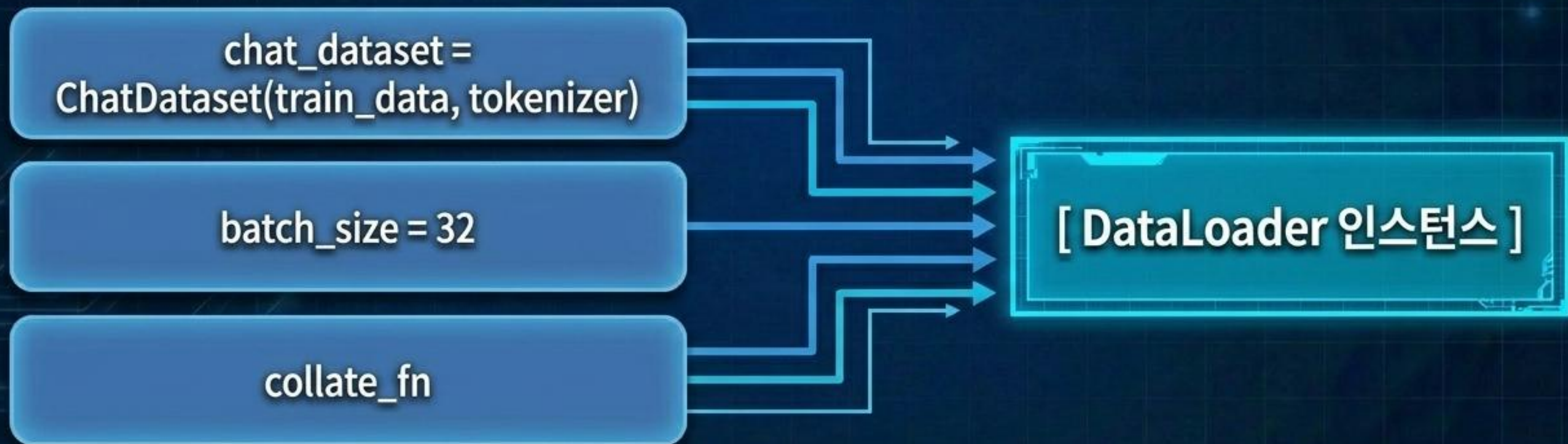

텐서 길이 정렬: `collate_fn` 및 패딩(Padding)

- 배치 처리를 위해 서로 다른 시퀀스 길이를 동일하게 정렬
- PyTorch의 `pad_sequence` 활용 (가장 긴 문장 기준)
- 빈 공간은 패딩 토큰(`pad_token_id=3`)으로 채움



조립 라인 가동: DataLoader 인스턴스 생성

- 데이터셋 클래스와 전처리 함수(collate_fn)를 하나로 통합
- 한 번에 처리할 샘플의 수(Batch Size)를 32로 고정하여 메모리 최적화



모듈 4: 파인튜닝 엔진 설정 (Optimizer & Steps)

- 옵티마이저: 안정적인 학습을 제공하는 Adam 알고리즘 채택
- 학습률(Learning Rate): 미세 조정을 위한 $3e-5$ 설정
- 총 스텝 수(Steps) 계산: 전체 데이터 개수 / 배치 사이즈 + 1

```
optimizer = torch.optim.Adam(model.parameters(),  
                               lr=3e-5, eps=1e-08)
```

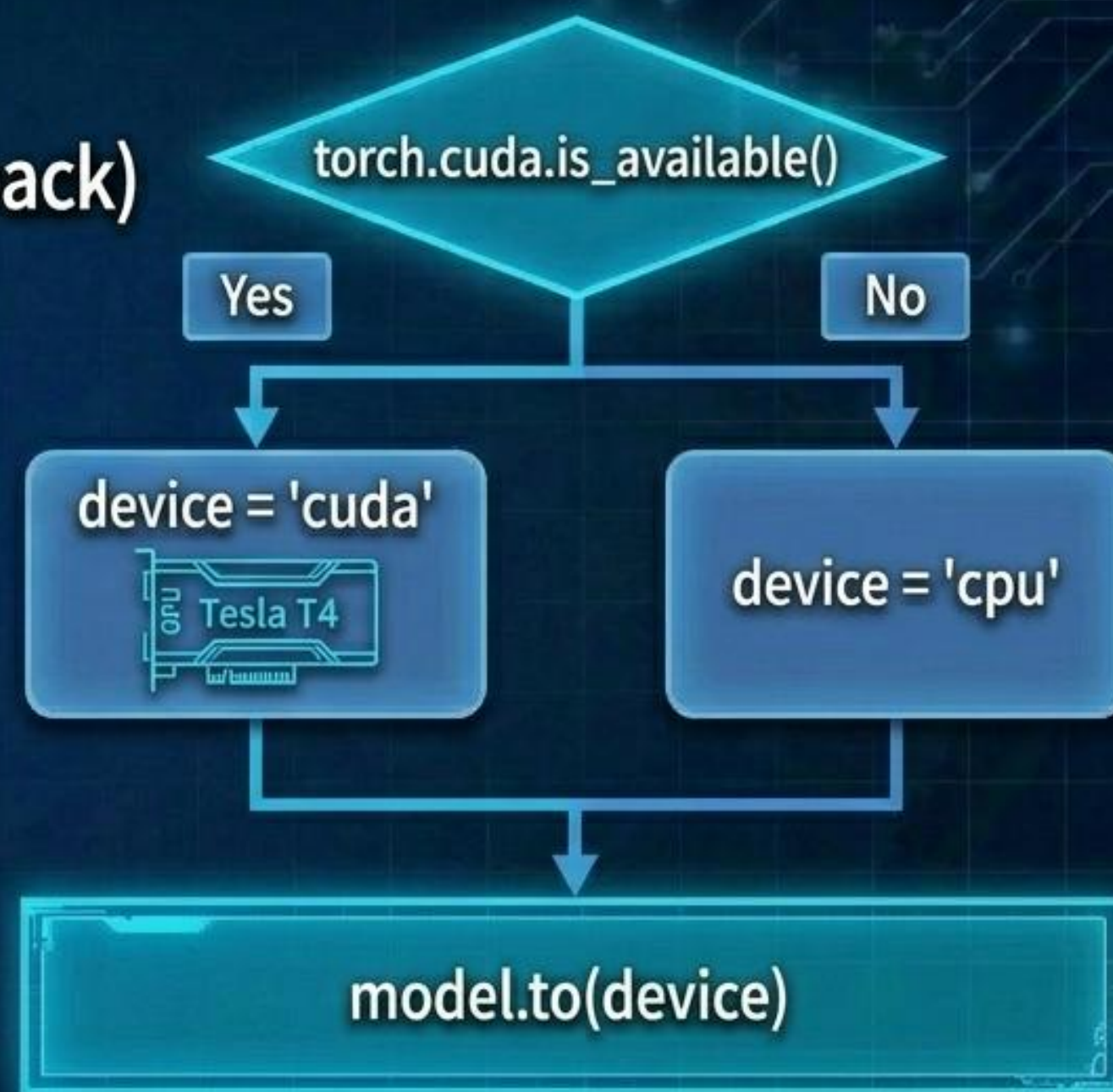
```
steps = len(train_data) // batch_size + 1
```

370

1 에포크당 총 370 스텝 진행

하드웨어 가속기(Device) 할당

- 모델 연산 최적화를 위해 GPU(cuda) 우선 할당
- GPU 사용 불가 시 CPU 환경으로 자동 폴백(Fallback)
- 결정된 디바이스로 트랜스포머 모델 객체 전송



학습 루프 메커니즘 (Training Loop Flow)

- 배치 단위로 입력 데이터 및 정답 레이블(Label) 할당
- 자기회귀(Autoregressive) 손실 계산을 위해 입력을 그대로 레이블로 활용
- 손실값(Loss) 역전파를 통한 모델 가중치 주기적 업데이트



파이토치(PyTorch) 에포크 단위 학습 로직

- 목표 에포크(EPOCHS = 3)만큼 전체 데이터 반복 학습
- tqdm 모듈을 통한 진행률 시각화 및 에포크별 누적 평균 오차(cost) 산출

```
1. batch = batch.to(device)
2. labels = batch.clone() // 입력을 그대로 레이블로 활용
3. optimizer.zero_grad()
4. result = model(input_ids=batch, labels=labels)
5. batch_loss.backward()
6. optimizer.step()
```


시스템 진단: 학습 경과(Cost) 모니터링

- 에포크가 반복됨에 따라 Cost(손실값)가 지속적으로 감소하는 양상 확인
- 모델이 주어진 챗봇 대화 패턴에 점진적으로 최적화됨을 증명

```
100%|██████████| 370/370 [01:41<00:00, 3.64it/s]  
[Epoch: 1] cost = 2.12702096  
100%|██████████| 370/370 [01:46<00:00, 3.48it/s]  
[Epoch: 2] cost = 1.69815452  
100%|██████████| 370/370 [01:46<00:00, 3.46it/s]  
[Epoch: 3] cost = 1.37507791
```


모듈 5: 챗봇 추론(Inference) 아키텍처

- 학습된 모델을 활용하여 새로운 사용자 입력에 대한 답변 생성
- 학습 당시의 데이터 포맷 규칙을 역으로 적용하는 전처리 필수



추론 1단계: 사용자 입력 전처리

- 사용자 문장 앞뒤로 구분 태그(<usr>, <sys>) 부착
- 생성 유도 관성: <sys> 토큰 뒤에 이어질 시스템 응답을 예측하도록 설계

text = '오늘도 좋은 하루!'

[BOS 토큰]

+

[<usr>]

+

[텍스트]

+

[<sys>]

```
input_ids = [tokenizer.bos_token_id] + tokenizer.encode('<usr>' + text + '<sys>')
```


추론 2단계: 텍스트 생성 및 응답 추출

- max_length=50 지정 및 early_stopping을 통한 문장 자동 종료 유도
- 출력된 전체 시퀀스에서 사용자 입력을 제외하고 <sys> 이후의 순수 답변만 분할(split) 추출

원시 출력(Raw Output)	</s><usr> 오늘도 좋은 하루!<sys> 오늘도 좋은 하루네요.</s>
분할 로직	 <pre>.split('<sys> ')[1].replace('</s>', '')</pre>
최종 추출(Final)	오늘도 좋은 하루네요.

추론 파이프라인 자동화: 사용자 정의 함수

- 전처리, 인코딩, 생성, 디코딩 과정을 하나의 캡슐화된 함수로 통합
- do_sample=True 및 top_k=2 인자를 활용하여 답변의 무작위성(다양성) 제어

```
def return_answer_by_chatbot(user_text):  
    (전처리 및 인코딩 생략)  
    output = model.generate(input_ids, max_length=50, do_sample=True, top_k=2)  
    chatbot_response = sentence.split('<sys> ')[1].replace('</s>', '')  
    return chatbot_response
```

다양성 제어 파라미터

라이브 테스트 1: 일상 대화 및 페르소나

- 일상적인 인사 및 정체성을 묻는 질문에 대한 적절한 맥락 파악 확인
- 단순한 단어 이어붙이기를 넘어서는 자연스러운 한국어 구사 능력 입증

안녕! 반가워~

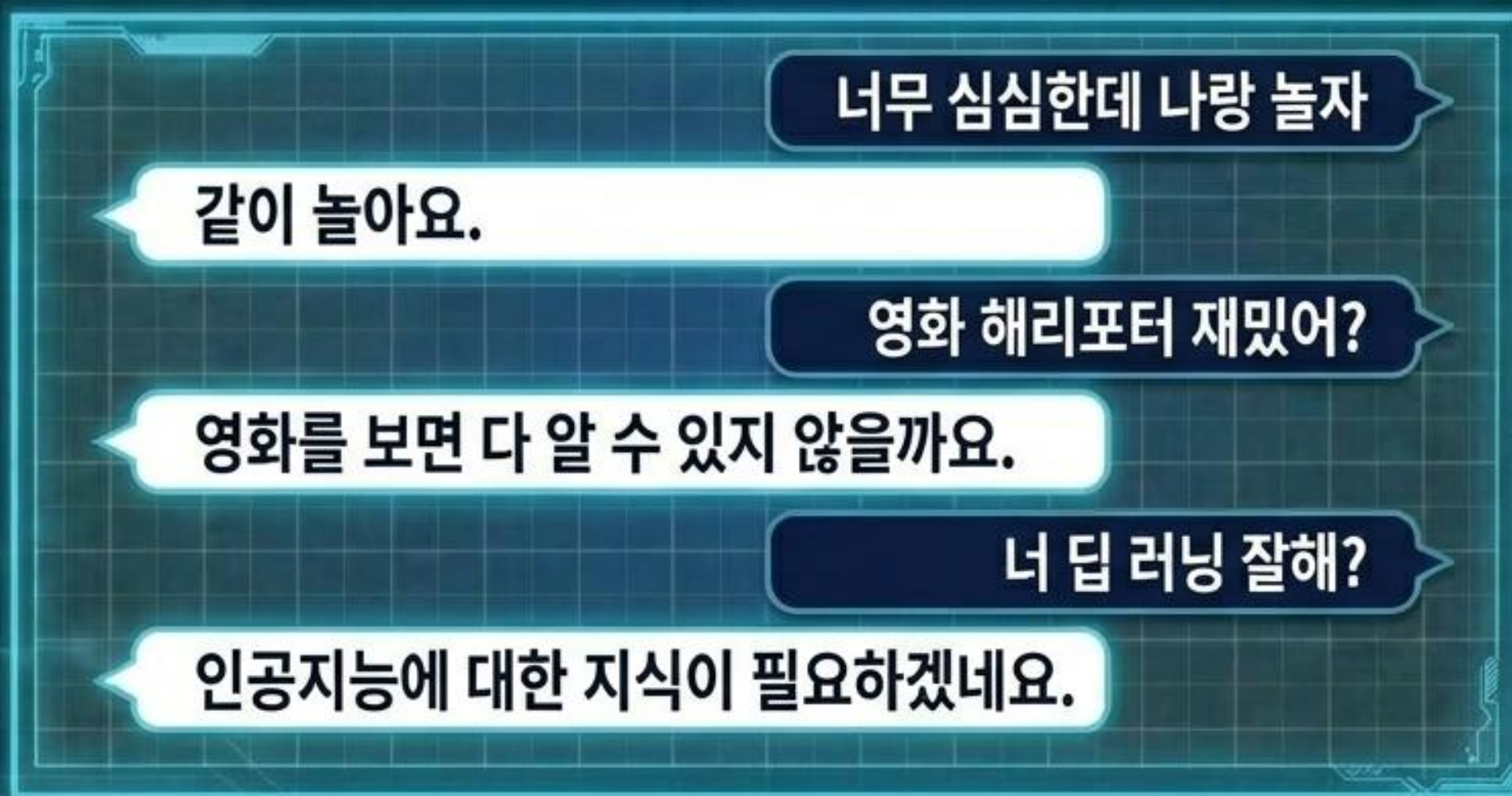
반가워요

너는 누구야?

저한테 물어보면 저랑 이야기할 수 있어요.

라이브 테스트 2: 맥락 대응 및 확장 과제

- 상황 대처(심심함), 특정 주제(영화), 전문 지식(딥러닝)에 대한 반응형 응답 완료
- KoGPT-2의 파라미터 한계를 극복하기 위한 향후 대규모 언어 모델(LLM) 파인튜닝 확장 필요성 시사



결론: 대화형 맥락 파악 성공 및 차세대 LLM 학습의 교두보 확보